

Inigo Quilez :: articles :: 3D axis aligned bounding boxes

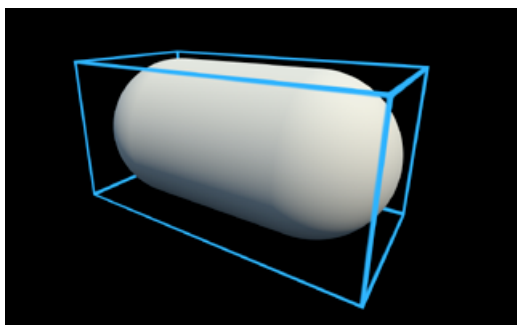


Intro

This is a list of 3D Axis Aligned Bounding Boxes (AABBs) for many of the SDF primitives in my [3D SDF functions](#). Computing bounding boxes is important for culling and pruning of primitives, which allows for faster rendering and physics of content made out of SDF shapes. For example, raymarching can be accelerated a big deal by only considering the primitives for which the bounding boxes overlap a tiled set of frustums.

The parameters of all functions below match exactly those of the [corresponding 3D SDFs](#). The return value of all these functions is a `bound3` datatype, which simply contains two `vec3` members with the minimum and maximum corners of the bounds in the X, Y and Z directions, ie, bounding box.

Note that all the primitives listed here come with a link to a realtime online demo in Shadertoy. In fact, this public playlist contains all the Shadertoy examples: <https://www.shadertoy.com/playlist/DXyczw>, so don't miss that out.



Segment

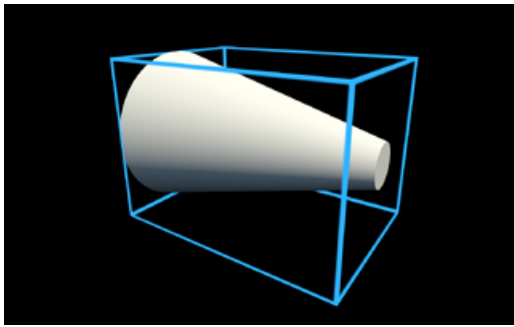
<https://www.shadertoy.com/view/tX2BDw>

```
bound3 aabbSegment( in vec3 pa, in vec3 pb, in float ra )
{
    vec3 a = pb - pa;
    return bound3( min(pa, pb) - ra,
```

```

        max(pa, pb) + ra );
    }

```



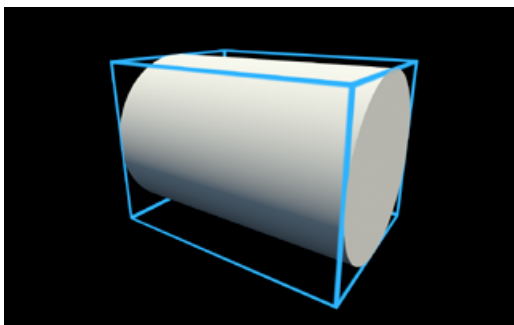
Cone

<https://www.shadertoy.com/view/WdjSRK>

```

bound3 aabbCone( in vec3 pa, in vec3 pb, in float ra, in float rb )
{
    vec3 a = pb - pa;
    vec3 e = sqrt(1.0-a*a/dot(a,a));
    vec3 ea = e*ra;
    vec3 eb = e*rb;
    return bound3( min(pa-ea, pb-eb),
                   max(pa+ea, pb+eb) );
}

```



Cylinder

<https://www.shadertoy.com/view/MtcXRf>

<https://iquilezles.org/articles/diskbbox>

```

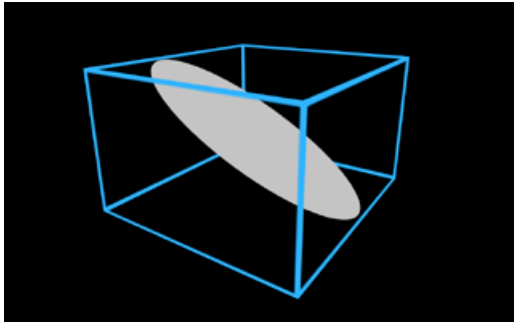
bound3 aabbCylinder( in vec3 pa, in vec3 pb, in float ra )
{
    vec3 a = pb - pa;
    vec3 e = ra*sqrt(1.0-a*a/dot(a,a));

```

```

    return bound3( min(pa, pb)-e,
                   max(pa, pb)+e );
}

```



Disk

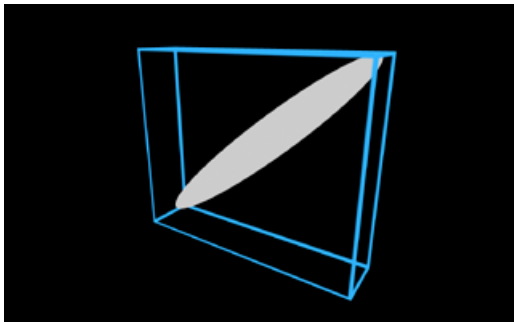
<https://www.shadertoy.com/view/ll3Xzf>

<https://iquilezles.org/articles/diskbbox>

```

bound3 aabbDisk( in vec3 ce, in vec3 no, in float ra )
{
    vec3 e = ra*sqrt(1.0-no*no);
    return bound3(ce-e, ce+e);
}

```



Ellipse

<https://www.shadertoy.com/view/t3jBWw>

<https://iquilezles.org/articles/ellipses>

```

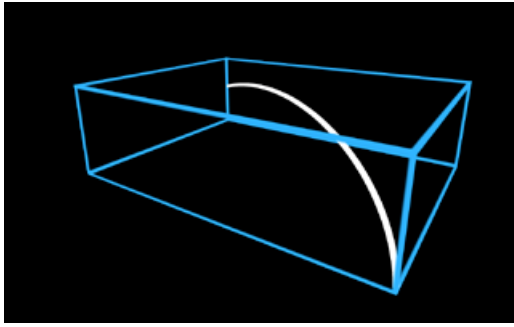
bound3 aabbEllipse( in vec3 ce, in vec3 au, in vec3 av )
{
    vec3 e = sqrt( au*au + av*av );
}

```

```

return bound3( ce-e, ce+e );
}

```



Quadratic Bezier

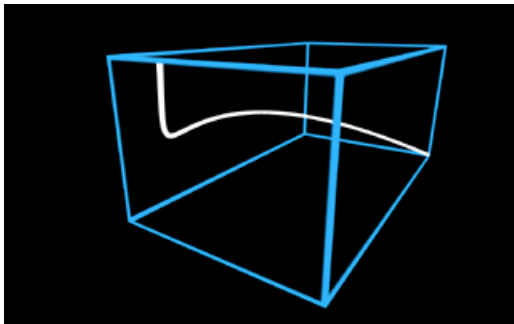
<https://www.shadertoy.com/view/tsBfRD>

<https://iquilezles.org/articles/bezierbbox/>

```

bound3 aabbBezier( in vec3 p0, in vec3 p1, in vec3 p2 )
{
    vec3 a = p0-2.0*p1+p2;
    vec3 b = p1-p0;
    vec3 t = clamp(-b/a,0.0,1.0);
    vec3 q = p0+t*(2.0*b+t*a);
    return bound3(min(min(p0,p2),q),
                  max(max(p0,p2),q));
}

```



Cubic Bezier

<https://www.shadertoy.com/view/MdKBWt>

<https://iquilezles.org/articles/bezierbbox/>

```

bound3 aabbBezier( in vec3 p0, in vec3 p1, in vec3 p2, in vec3 p3 )
{
    vec3 c = -p0+      p1;

```

```
vec3 b = p0-2.0*p1+ p2;  
vec3 a = -p0+3.0*p1-3.0*p2+p3;  
vec3 g = sqrt(max(b*b-a*c,0.0));  
vec3 t1 = clamp((-b-g)/a,0.0,1.0);  
vec3 t2 = clamp((-b+g)/a,0.0,1.0);  
vec3 q1 = p0+t1*(3.0*c+t1*(3.0*b+t1*a));  
vec3 q2 = p0+t2*(3.0*c+t2*(3.0*b+t2*a));  
return bound3(min(min(p0,p3),min(q1,q2)),  
              max(max(p0,p3),max(q1,q2)));  
}
```

inigo quilez - learning computer graphics since 1994